

Modül 06: Kalıtım ve Çok Biçimlilik

Kalıtım, Java'nın 1995'te üzerinden pazarlandığı özellik ve modern Java'nın ölçülü kullanmanızı tavsiye ettiği özellik. İkisi de doğru ve bu modül nedenini anlatıyor.

Her gün kullanacağınız kısım aslında `extends` değil. `equals` / `hashCode` sözleşmesi, ve bu modül onu anlatmak yerine kıırarak gösteriyor.

Neler Öğreneceksiniz

- Dinamik gönderim ve kalıtımın asıl amacının bu olduğu
- `super` ve `@Override` yazmanın her seferinde neden değdiği
- Tasarımınız hakkında zıt talimatlar olarak `abstract` ve `final`
- `equals` / `hashCode` sözleşmesi, bir `HashSet` kıırarak gösterilmiş
- Çok biçimli *olmayanlar*: alanlar ve statik metotlar

Başka Bir Dilden Geliyorsanız

	Python	Java
Kalıtım	<code>class Dog(Animal)</code>	<code>class Dog extends Animal</code>
Üst sınıfı çağırma	<code>super().__init__()</code>	<code>super(...)</code> , ve önce gelmeli
Çoklu kalıtım	<code>var</code>	yok, yalnızca tek. Arayüzler boşluğu dolduruyor.
Soyut	<code>abc.ABC</code>	<code>abstract</code> anahtar kelimesi
Alt sınıflamayı engelleme	pek mümkün değil	<code>final</code>
Eşitlik	<code>__eq__</code> ve <code>__hash__</code>	<code>equals</code> ve <code>hashCode</code> , aynı ikili

Java yalnızca bir üst sınıfa izin verir. Arayüzlerin var olma sebebi bu kısıt ve Modül 07 onları anlatıyor.

Ders

Dinamik gönderim

Tek bir çağrı yeri, üç metot. `Inheritance.java` dosyasından:

```
[log] system reboot
[email to ada@example.com] system reboot
[sms to +44 7700 900000] system reboot
```

Döngü `Notification` tipine göre yazıldı. Derleyici yalnızca o tipin bir `deliver()` metodu *olduğunu* denetler. Hangisinin çalışacağına gerçek nesneden, çalışma zamanında karar verilir.

Pratik kazanç: o döngüye hiç dokunmadan dördüncü bir bildirim tipi ekleyebilirsiniz. Temel tipe göre yazılmış kod çalışmaya devam eder.

super ve @Override

`super(...)` üst sınıf kurucusunu çağırır ve Modül 05'in anlattığı istisna dışında önce gelmelidir.

`super.method()` üst sınıfın sürümünü çağırır, davranışı değiştirmek yerine genişletmenin yolu budur.

@Override her zaman yazın. Süs değil, derleyiciye gerçekten bir şeyi geçersiz kıldığınızı denetlemesi için verilen bir talimattır. `OverrideTypo.java` `deliver` yerine `delivar` yazıyor:

```
error: delivar() in Email does not override or implement a method from a supertype
@Override
^
```

Anotasyon olmadan bu kusursuz derlenir, kimsenin çağırmadığı bir metod ekler ve temel uygulamayı çalışmaya devam ettirir. Kod doğru görüldüğü için hata incelemekten sağ çıkar. Tek bir anotasyon tüm kategoriyi ortadan kaldırıyor.

abstract ve final

`abstract` alt sınıflar bunu doldurmalı der. `final` kimse bunu değiştiremez der.

`AbstractAndFinal.java` dosyasından:

```
abstract class Shape {
    abstract double area();

    final boolean isLargerThan(Shape other) {
        return this.area() > other.area();
    }
}
```

Soyut bir sınıf kurulamaz, ki amaç budur: belirli bir şekli olmayan bir şeklin anlamlı bir alanı yoktur. Arayüzden farklı olarak sıradan metotlar ve durum da tutabilir.

Metotta `final` geçersiz kılmayı engeller. Sınıfta `final` genişletmeyi tamamen engeller. `String` ve `Integer` ikisi de final, ve bu yardımseverlikten uzaklık değil: `String` sınıfını alt sınıflayıp `equals` metodunu geçersiz kılabilseydi herhangi biri, hiçbir yerde hiçbir kod bir `String` değerine güvenemezdi.

Sınıfları doğal hissettiğinden daha sık final yapın. Genişletilmek üzere tasarlanmamış bir sınıf genellikle güvenle genişletilemez, çünkü alt sınıf herhangi bir metodu geçersiz kılıp sınıfın dayandığı değişmezleri bozabilir. Modül 09, hepsi ya da hiçbiri arasındaki orta yol olan `sealed` kavramını tanıtıyor.

equals/hashCode sözleşmesi

Asıl karşılaşacağınız kısım bu. `EqualsHashCodeContract.java` dosyasından, `equals` metodunu geçersiz kılıp `hashCode` metodunu unutan bir sınıf:

```
two equal points added, set size: 2
contains an equal point:           false
but the two ARE equal:            true
```

Tekrar okuyun. Küme, eşit olduğunu kabul ettiği iki öğeyi tutuyor ve içinde zaten bulunan bir öğeye eşit olanı bulamıyor.

`HashSet` her şeyin üzerinde `equals` çağırır. Bir kova seçmek için `hashCode` hesaplar ve yalnızca o kovanın içinde karşılaştırır. `Object.hashCode` kimliğe dayalıdır, dolayısıyla iki ayrı örnek `equals` ne derse desin farklı kod alır, farklı kovalara düşer ve hiç karşılaşmaz.

`hashCode` ekleyin, düzgün davranır:

```
two equal points added, set size: 1
contains an equal point:           true
```

Sözleşmenin tamamı:

1. `a.equals(b)` ise `a.hashCode() == b.hashCode()` olmalı. İşleri bozan bu.
2. Eşit hash kodları eşitlik anlamına gelmez. Çakışmalar meşrudur.
3. Yansılmalı: `a.equals(a)`
4. Simetrik: `a.equals(b) == b.equals(a)`
5. Geçişli: `a=b` ve `b=c` ise `a=c`
6. Tutarlı: hiçbir şey değişmediyse her seferinde aynı cevap
7. `a.equals(null)` her zaman false

`Objects.hash(...)` kullanın ve `equals` metodunun karşılaştırdığı alanların **tam olarak** aynısını geçirin. Bir alanı birinde kullanıp diğerinde kullanmamak, birinci kuralı `hashCode` metodunu tamamen atlamak kadar bozar.

`toString` ailenin üçüncü üyesi. Sözleşmenin parçası değil, ama okunaklı bir log satırı ile `BadPoint@4034c28c` arasındaki fark.

Kısayol: bir record üçünü de bileşenlerinden doğru biçimde yazar. Modül 08 onları anlatıyor, ve veri taşıyan bir sınıf için record neredeyse her zaman daha iyi cevap.

Çok biçimli olmayanlar

Metotlar nesneye göre gönderilir. Alanlar ve statikler gönderilmez. `WhatIsNotPolymorphic.java` dosyasında nesne bir `Child`, ama `Parent` değişkeninde tutuluyor:

```
instance method: Child.instanceName
field access:    parent field
as a Child:      child field
static method:   Parent.staticName
```

Her iki alan da nesnede aynı anda var. Alt sınıf üst sınıfın alanını değiştirmede, onu gizleyen ikinci bir alan ekledi. Statik metotlar da benzer şekilde *gizlenir*, geçersiz kılınmaz.

Buradan çıkan kural: **asla bir alanı gölgelemeyin, asla üst sınıfın statik metodunu yeniden bildirmeyin.** İkisi de yasaldir, ikisi de görüldüğü işi yapmaz ve ikisini de yazmak için iyi bir sebep yoktur.

Modül 04'ün `static` kelimesinin sizi sonradan engellediği uyarısının sebebi de bu. Statik bir metot geçersiz kılınmaz, dolayısıyla buradaki hiçbir şeye katılamaz.

Çalıştırın

```
java modules/06-inheritance-and-polymorphism/examples/Inheritance.java
java modules/06-inheritance-and-polymorphism/examples/AbstractAndFinal.java
java modules/06-inheritance-and-polymorphism/examples/EqualsHashCodeContract.java
java modules/06-inheritance-and-polymorphism/examples/WhatIsNotPolymorphic.java

# Bilerek başarısız olur.
java modules/06-inheritance-and-polymorphism/examples/OverrideTypo.java

./scripts/verify-examples.sh modules/06-inheritance-and-polymorphism
```

Sık Yapılan Hatalar

`equals` metodunu `hashCode` olmadan geçersiz kılmak. Nesneleriniz bir `HashMap` içine girer ve bir daha çıkmaz. Bu modüldeki en yaygın ciddi hata.

`equals` metodunun yok saydığı bir alanı `hashCode` içine koymak ya da tersi. Aynı bozukluk, fark edilmesi daha zor.

`equals` ve `hashCode` içinde değiştirilebilir bir alan kullanmak. Nesneyi bir kümeye koyun, alanı değiştirin, artık kendi koleksiyonunun içinde kaybolmuştur, çünkü yeni hash kodunun eşleşmediği bir kovada dosyalanmıştır.

`@Override` yazmamak. Bedava hata tespitini reddetmek.

Kurucudan geçersiz kılınabilir bir metot çağırmak. Alt sınıfın geçersiz kılma metodu, alt sınıf kurucusu alanlarını ilklendirmeden önce çalışır, dolayısıyla null ve sıfır görür. Böyle metotları `final` ya da `private` yapın.

Kod tekrar kullanmak için kalıtıma uzanmak. `extends` "bir çeşididir" demektir, "ondan ödünç alır" değil. Alt sınıf dürüstçe üst sınıfın kullanıldığı her yerde kullanılmıyorsa kompozisyon kullanın: diğer nesneyi bir alan olarak tutun.

Özet Çıkarımlar

- **Dinamik gönderim metodu nesneden seçer**, değişkenin tipinden değil.
- `@Override` her zaman yazın. Sessiz hataları derleme hatalarına çevirir.
- `abstract` alt sınıfları doldurmaya zorlar. `final` değişikliği yasaklar. Varsayılan olarak `final` tercih edin.
- `equals` ve `hashCode` metotlarını her zaman birlikte, aynı alanlar üzerinde geçersiz kılın.
- **Asla alanları gölgelemeyin ya da statik metotları yeniden bildirmeyin.** Nesneye göre değil bildirilen tipe göre çözümlenirler.
- **"Bir çeşididir" ilişkisi dürüst değilse kompozisyonu tercih edin.**

Ödev

[homework/README.md](#)

Bir `HashSet` kümesini bilerek kırın, teşhis edin, sonra düzeltin. Referans çözüm [solutions/06-inheritance-and-polymorphism/](#) altında.